

C-HASH Self-Assessment Test Report

x86 Architecture

Version 20260630-v5

C Hash Team

Next-generation Commercial Cryptographic Algorithms Program (NGCC)

Assessment Date: 2026-06-30

Contents

1 Basic Algorithm Information 2

1.1 Algorithm Architecture 2

2 Assessment Environment 2

2.1 Hardware Platform 2

2.2 Instruction Set Extensions 2

2.3 Software Platform 3

2.4 Build Configurations 3

3 Compilation Results 4

4 Functional Testing (KAT Verification) 4

4.1 Test Vector Generation 4

4.2 KAT Verification Results 4

4.3 Loop Test Verification 4

4.4 Cross-Version Output Consistency 5

5 Performance Testing 5

6 Resource Consumption Testing 6

7 Summary 7

8 Raw Evidence Index 7

1 Basic Algorithm Information

Attribute	Value
Algorithm Category	Cryptographic Hash Algorithm
Algorithm Name	C-HASH
Algorithm Instances	C-HASH-512 (512-bit digest) C-HASH-1024 (1024-bit digest)
Implementation Versions	Reference Implementation (REF) Performance Optimization (OPT) Resource Optimization (ADD)

Table 1: Basic Algorithm Information

1.1 Algorithm Architecture

C-HASH is a cryptographic hash function built on a two-layer architecture:

1. **C-Engine:** A 1536-bit permutation consisting of 28 rounds of Substitution-Permutation-Substitution (SPS) structure. Each round comprises an S-box layer, an M-box (mixing) layer, and a round-constant addition layer.
2. **Mode Layer:** A unified Variable Input Length (VIL) and Finalization (FIL) structure shared identically between C-HASH-512 and C-HASH-1024.

The two algorithm instances differ in only four aspects:

1. **C-Engine variant:** C-HASH-512 uses pure permutation $P(x)$; C-HASH-1024 uses FeedForward permutation $P_{FF}(x) = P(x) \oplus x$.
2. **Lane ID:** C-HASH-512 uses lane IDs 0x01/0x02/0x03; C-HASH-1024 uses 0x81/0x82/0x83.
3. **IV encoding:** The initialization vector encodes the hash length in bytes 88–91 as big-endian.
4. **Output length:** 512-bit or 1024-bit digest via leftmost-bit extraction.

2 Assessment Environment

2.1 Hardware Platform

2.2 Instruction Set Extensions

The CPU supports the x86-64 base instruction set with the following key extensions: FPU, TSC, MSR, PAE, CX8, APIC, SEP, MTRR, PGE, MCA, CMOV, PAT, PSE36, CLFLUSH, MMX, FXSR, SSE, SSE2, SS, HT, SYSCALL, NX, RDTSCP, LM, PCLMULQDQ, SSSE3, FMA, CX16, PCID, SSE4.1, SSE4.2, X2APIC, MOVBE, POPCNT, AES, XSAVE, AVX, F16C, RDRAND, BMI1, **AVX2**, BMI2, INVPCID, AVX512F, AVX512DQ, RDSEED, ADX, SMAP, CLFLUSHOPT, CLWB, AVX512CD, AVX512BW, AVX512VL.

Key extensions utilized by the Optimized implementation:

- **AVX2:** 256-bit integer SIMD for parallel S-box lookup
- **BMI1/BMI2:** Bit manipulation instructions

Component	Specification
CPU Vendor	Intel Corporation
CPU Model	Intel Xeon Platinum (Cascade Lake/Ice Lake generation)
CPU Architecture	x86-64 (64-bit, little-endian)
Base Clock Frequency	2,500.004 MHz (2.5 GHz)
CPU L3 Cache	33,792 KB (33 MB, shared)
Physical Cores	2
Cores Used for Testing	1 (single-core mode)
Memory Type	DDR4 ECC
Total Physical Memory	4.0 GiB
Available Memory at Test	3.1 GiB

Table 2: Hardware Platform Specifications

Component	Specification
Operating System	Debian GNU/Linux 12 (bookworm)
OS Kernel	Linux 5.10.134-18.0.11.lifsea8.x86_64
Kernel Build Date	2026-05-19
Distribution	Debian 12.0 (stable)
C Compiler	GNU GCC 12.2.0-14+deb12u1
C Standard	ISO/IEC 9899:1999 (C99)
Build Tool	GNU Make (implicit)

Table 3: Software Platform Specifications

2.3 Software Platform

2.4 Build Configurations

Per the Guidelines, each version uses the following compiler flags:

Version	Build Flags
REF	-O2 -std=c99 -Wpedantic -Wall -Wextra
OPT	-O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra
ADD	-O0 -march=x86-64 -mavx2 -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra

Table 4: Build Configurations

Flag descriptions:

- `-std=c99`: ISO C99 standard compliance
- `-Wpedantic`: Strict ISO C99 compliance enforcement
- `-Wall`: Standard compiler warning set
- `-Wextra`: Extended compiler warning set
- `-O2`: Moderate optimization (REF)

- `-O3`: Aggressive optimization with auto-vectorization (OPT)
- `-Os`: Size optimization (ADD)
- `-march=x86-64`: Target x86-64 architecture
- `-mavx2`: Enable AVX2 instruction set
- `-mtune=native`: Tune for host CPU microarchitecture
- `-flto`: Link-Time Optimization for cross-module inlining
- `-fomit-frame-pointer`: Free RBP register for general use

3 Compilation Results

All six instances were compiled as shared libraries using the build configurations specified in Table 4.

Instance	Result	Errors	Warnings
C-HASH-512 REF	Success	0	1
C-HASH-512 OPT	Success	0	3
C-HASH-512 ADD	Success	0	1
C-HASH-1024 REF	Success	0	1
C-HASH-1024 OPT	Success	0	2
C-HASH-1024 ADD	Success	0	1

Table 5: Compilation Results

All six instances compiled successfully with zero errors. Warnings were generated by `-Wall` `-Wextra` flags and relate to standard coding practice recommendations (e.g., unused parameters in API wrapper functions); these warnings do not affect correctness or functionality.

4 Functional Testing (KAT Verification)

4.1 Test Vector Generation

Known Answer Test (KAT) vectors were generated using the official Deterministic Random Number Generator (DRNG) with fixed seeds per the Submission Requirements:

- **KAT_2_12**: 4,097 vectors per instance (message lengths 0 to $2^{12} = 4,096$ bits), seed “KAT_2_12”
- **KAT_2_23**: 2 vectors per instance (message length $2^{23} = 8,388,608$ bits), All-0 and All-1 fill
- **KAT_Loop**: Loop Test on $2^{13} = 8,192$ -bit messages with 1,000,000 iterations, seed “KAT_Loop”

4.2 KAT Verification Results

4.3 Loop Test Verification

The Loop Test was executed with 1,000,000 iterations on 8,192-bit messages for all six instances. The Optimized implementation’s output was verified against the Reference-generated KAT file:

Overall Functional Test: ALL PASS (24,594 total vector tests, zero failures).

Instance	KAT Type	Vectors	Pass	Result
C-HASH-512 REF	KAT_2_12	4,097	4,097	PASS
	KAT_2_23	2	2	PASS
C-HASH-512 OPT	KAT_2_12	4,097	4,097	PASS
	KAT_2_23	2	2	PASS
C-HASH-512 ADD	KAT_2_12	4,097	4,097	PASS
	KAT_2_23	2	2	PASS
C-HASH-1024 REF	KAT_2_12	4,097	4,097	PASS
	KAT_2_23	2	2	PASS
C-HASH-1024 OPT	KAT_2_12	4,097	4,097	PASS
	KAT_2_23	2	2	PASS
C-HASH-1024 ADD	KAT_2_12	4,097	4,097	PASS
	KAT_2_23	2	2	PASS

Table 6: KAT Verification Results

Instance	KAT Match	Result
C-HASH-512 OPT	Full bitwise match with REF-generated KAT	PASS
C-HASH-1024 OPT	Full bitwise match with REF-generated KAT	PASS

Table 7: Loop Test Verification

4.4 Cross-Version Output Consistency

All three versions (REF/OPT/ADD) were cross-verified on 34 message lengths covering:

- LDRH domain boundary: 1,471 / 1,472 / 1,473 bits
- Block boundaries: 511/512/513, 735/736/737, 1,023/1,024, 4,095/4,096, 8,191/8,192 bits
- Non-byte-aligned lengths: 1, 7, 9, 15, 17, 63, 65, 127, 129, 255, 257 bits

All 34 test cases produced **identical digests** across all three versions for both C-HASH-512 and C-HASH-1024 (204 instance-level comparisons total), confirming strict output consistency.

5 Performance Testing

Performance was measured via high-resolution timer (`clock_gettime(CLOCK_MONOTONIC)`) using the S1–S8 data levels specified in the Guidelines. Each measurement includes warmup iterations followed by the specified number of timed iterations. CPU clock cycles are calculated at the nominal frequency of 2.5 GHz.

Notes:

- S1–S4 are processed in LDRH (short-message) mode; S5–S8 are processed in VIL (long-message) mode.
- The throughput spike at S3/S4 (512B/1KiB) for OPT reflects the efficiency of AVX2 vectorization when the message fits within the L1 cache.
- The dip at S5/S6 for 512-OPT reflects the transition from LDRH to VIL processing overhead.
- Iteration counts: S1: 50,000; S2: 20,000; S3: 5,000; S4: 1,000; S5–S6: 500; S7: 200; S8: 100.

Instance	Metric	S1	S2	S3	S4	S5	S6	S7	S8
		32B	128B	512B	1KiB	4KiB	8KiB	16KiB	64KiB
512-REF	MB/s	0.05	0.19	0.76	1.55	1.58	1.75	2.03	2.19
	us/call	79.80	79.78	80.75	78.80	309.08	557.43	961.96	3567.59
	cycles	199489	199440	201886	197010	772689	1393577	2404912	8918974
512-OPT	MB/s	1.01	4.07	16.33	32.41	45.83	43.35	64.15	71.09
	us/call	3.77	3.75	3.74	3.77	10.66	22.53	30.45	109.89
	cycles	9415	9364	9342	9418	26638	56322	76113	274733
512-ADD	MB/s	0.03	0.12	0.47	0.94	0.95	1.05	1.21	1.31
	us/call	132.73	131.73	130.79	129.47	512.47	934.34	1618.34	5954.66
	cycles	331815	329330	326966	323686	1281186	2335860	4045851	14886655
1024-REF	MB/s	0.05	0.19	0.76	1.53	1.55	1.76	2.01	2.14
	us/call	80.77	80.29	79.86	79.55	314.68	554.34	973.47	3646.07
	cycles	201928	200724	199650	198879	786712	1385840	2433679	9115163
1024-OPT	MB/s	1.02	4.06	16.24	32.23	45.76	54.47	63.32	71.13
	us/call	3.76	3.76	3.76	3.79	10.67	17.93	30.85	109.83
	cycles	9388	9389	9393	9469	26674	44817	77114	274572
1024-ADD	MB/s	0.03	0.12	0.47	0.96	0.95	1.10	1.23	1.33
	us/call	129.47	128.97	129.67	127.62	511.64	886.27	1583.91	5871.04
	cycles	323674	322435	324167	319052	1279089	2215669	3959768	14677601

Table 8: Performance Test Results (S1–S8 Data Levels)

6 Resource Consumption Testing

Static memory was measured as the sum of text (code), data (initialized), and bss (uninitialized) segments of the compiled `c-hash.o` object file (compiled without `-fplt` for accurate per-module sizing).

Instance	Text (B)	Data (B)	BSS (B)	Total (B)
C-HASH-512 REF	3,952	0	0	3,952
C-HASH-512 OPT	26,564	0	1,216	27,780
C-HASH-512 ADD	2,255	0	0	2,255
C-HASH-1024 REF	4,504	0	0	4,504
C-HASH-1024 OPT	29,540	0	1,216	30,756
C-HASH-1024 ADD	2,608	0	0	2,608

Table 9: Static Memory Consumption

Observations:

- ADD achieves the smallest code footprint (2,255 B for 512, 2,608 B for 1024), approximately $0.57\times$ of REF.
- OPT has the largest footprint due to AVX2 lookup tables (512 B each for S-box, S-box+SubConst fused) and unrolled loop code, but remains under 31 KB.
- The 1,216-byte BSS in OPT versions corresponds to the 32-byte aligned AVX2 state buffers and lookup tables.
- Peak runtime memory depends on maximum message size: for KAT_2_33 (2^{33} bits $\approx$ 1 GB), approximately 1 GB of available memory is required for message storage plus working buffers ($\approx$ 2 KB).

7 Summary

Assessment Item	Result
Compilation (6 instances)	All success, 0 errors
KAT_2_12 (4097 vectors $\times$ 6)	24,582/24,582 PASS
KAT_2_23 (2 vectors $\times$ 6)	12/12 PASS
KAT_Loop (1 vector $\times$ 2)	Full bitwise match PASS
Cross-Version Consistency (34 cases $\times$ 2)	0 mismatches
512-OPT Peak Throughput	71.09 MB/s (S8/64KiB)
1024-OPT Peak Throughput	71.13 MB/s (S8/64KiB)
OPT Speedup vs REF (S8)	32.4$\times$
ADD Code Size vs REF	0.57$\times$

Table 10: Assessment Summary

Key Findings:

1. All six instances compile successfully with zero errors and demonstrate full functional correctness across 24,594 KAT vectors.
2. The Optimized implementation achieves **71+ MB/s** peak throughput on 64 KiB messages at 2.5 GHz, representing a **32.4 $\times$** speedup over the Reference implementation.
3. The Resource-Optimized implementation achieves the smallest code footprint (2,255 bytes for C-HASH-512), approximately **0.57 $\times$** of the Reference.
4. All three versions produce **bitwise-identical outputs** across all test cases, confirming strict algorithm correctness independent of optimization level.
5. The VIL/FIL mode layer is **unified and identical** across 512 and 1024 instances; FeedForward in C-HASH-1024 occurs entirely within the C-Engine-FF permutation.

8 Raw Evidence Index

Metric	Source	Description
Compilation	Shell commands	gcc with version-specific flags
KAT_2_12	Generated vectors	DRNG-seeded messages 0–4096 bits
KAT_2_23	Generated vectors	All-0/All-1 messages 2^{23} bits
KAT_Loop	Generated vectors	1,000,000 iterations on 2^{13} bits
Performance	Python timing script	<code>time.perf_counter()</code> with warmup
Static Memory	<code>size</code> command	Object file segment analysis

Table 11: Raw Evidence Index

All test data, source code, and intermediate results are available in the submission package under the `Test_Vectors/` and `Implementations/` directories.